Semestrální projekt

NI-PDP 2023/2024: Paralelní algoritmus pro řešeni problému

Úloha VJS: výměna jezdců na šachovnici

Andrej Ohrablo

magisterské studium, FIT CVUT, Thákurova 9, 160 00 Praha 6

May 8, 2024

1 Definice problému a popis sekvenčního algoritmu

Zadanie problému prebrané zo stránok predmetu:

Pravidla a cíl hry:

- **Počáteční stav** je dán pozicemi bílých (nacházejí se v oblasti W) a černých (nacházejí se v oblasti B) jezdců.
- Každý jezdec se na šachovnici pohybuje podle šachových pravidel (tah ve tvaru L na volné políčko).
- **Cílem hry** je střídáním tahů bílých a černých jezdců převést počáteční stav na cílový, ve kterém bílí jezdci obsadí oblast B a černí jezdci obsadí oblast W.
- Protože k = |B| = |W|, jsou oblasti B a W na počátku vždy plně obsazeny.
- Střídání barev tahů jezdců se přestane provádět, pokud všichni jezdci jedné barvy jsou již ve své cílové oblasti a tahy provádějí pouze jezdci opačné barvy.

Úkol:

Nalézt **nejkratší posloupnost** tahů vedoucí z počátečního stavu do cílového stavu.

Výstup algoritmu:

- Číslo udávající minimální počet tahů po realizaci výměny a výpis posloupnosti tahů vedoucí z počátečního stavu do cílového stavu.
- Tah vypište jako stav celé šachovnice po jeho provedení. [1]

Sekvenčný algoritmus je typu BB-DFS s neobmedzenou hĺbkou. Preto vzniká potreba nastaviť si hornú mez maximálneho počtu ťahov. Tá bola už v zadaní určená ako súčet vzdialeností medzi počiatočnými pozíciami jazdcov a najvzdialenejšími políčkami v cieľovej oblasti. Pri nájdení riešenia s nižším počtom ťahov sa horná mez upravila na hodnotu riešenia. Až na ojedinelé výnimky pri veľmi jednoduchých inštanciách kedy bola nastavení príliš nízko sa ukázala ako dostačujúca.

Na včasné dobehnutie algoritmu bolo potrebné vetve DFS ukončovať skorej ako pri maximálnom počte krokov. Zadanie preto špecifikovalo aj donú mez každého medzistavu ako súčet počtu ťahov pre každého jazdca do najbližšieho políčka v cieľovej oblasti. Beh vetve DFS sa následne ukončoval podľa nasledujúceho vzroca: *dosavadni_pocet_tahu(S) + dolná_mez(S')+1 >= současná horní mez.*

Podľa odporúčaní na cvičeniach bol do riešenia pridaný BFS algoritmus na predpočítanie 2D mapy s počtom ťahov z každého bodu x do každého bodu y. Táto úprava mala pomerne vysoký dopad na rýchlosť výpočtu dolnej meze.

Ukladanie najlepšieho počtu ťahov bolo riešené cez globálnu premennú, čo sa ukázalo ako najlepší spôsob ako oznamovať nové hodnoty riešení naprieč všetkými bežiacimi vetvami DFS. Cesta najlepšieho riešenia bola pri sekvenčnom riešení riešená ako návratová hodnota behu DFS.

Algoritmus dobehol pri nájdení riešenia rovnému dolnej mezi štartovacieho stavu alebo pri dobehnutí všetkých vetiev DFS. Na testovaných inštanciách bežal dostatočne rýchlo a nebolo potrebné vytvárať ďalšie úpravy. Najväčší vplyv na rýchlosť malo asi očakávane skoršie ukončovanie DFS vetiev.

Vstup programu (in_0010.txt):

```
4 5 3 3
0 0 0 2
3 2 3 4
```

Výstup programu:

```
Step 0
  0 1 2 3
0 W . . .
1 W . . .
2 W . . B
3 . . . B
4 . . . B
```

```
Step 14
  0 1 2 3
0 B . . .
1 B . . .
2 B . . W
3 . . . W
4 . . . W
```

Optimal number of turns: 14

Časy sekvenčného riešenia na lokálnom PC (časy na clusteri boli ~5.5krát pomalšie)

Vstup	**Čas**
In_0001.txt	1s
In_0002.txt	0,2s
In_0010.txt	33s
In_0012.txt	2s
In_0014.txt	18s
In_0015.txt	15s
In_0016.txt	7s

2 Popis paralelniho algoritmu a jeho implementace v OpenMP - taskovy paralelismus

Paralelné riešenie vychádza vo veľkej miere z toho sekvenčného. Jedinou podstatnou zmenou bolo pridanie OpenMP knižnice a použitie správnych príkazov na spustenie paralelného výpočtu.

Zapnutie dfs výpočtu je obalené pragma príkazmi pre jeden paralelný výpočet riešenia:

```
# pragma omp parallel
   {
   #pragma omp single
     {
         DFS();
    }
  }
```

V rekurzívnom volaní DFS sa pre každý nový stav vytvorí nový task, ktorý si voľné vlákno môže prebrať. Pre ďalšie (veľmi mierne) zrýchlenie sa v istej hĺbke už nevytvára nový task ale vlákno už dopočíta danú DFS vetvu sekvenčne:

```
if(depth < 2) {
      #pragma omp task
       {
        DFS();
       }
       else{
          DFSSEQ();
        }
}
```

Na konci funkcie je pre synchronizáciu pridaný príkaz *#pragma omp taskwait*.
Kvôli predávaniu si vetiev DFS funkcie medzi vláknami z nej boli odstránené návratové hodnoty. Cesta najkratšieho riešenia sa tak stala globálnou premennou.

Zmena najlepšieho riešenia je pre ochranu prístupu obalená príkazom *#pragma omp critical*.

```
#pragma omp critical
   {
      if (riešenie < minule_riešenie) {
         minulé_riešenie = riešenie;
         cesta = nová_cesta;
       }
    }
```

3 Popis paralelniho algoritmu a jeho implementace v OpenMP - datovy paralelismus

Riešenie pre dátový paralelizmus opäť vychádza zo sekvenčného riešenia a je typu master-slave. Najväčší rozdiel je na samotnom začiatku počítania, kedy jedno vlákno počíta sekvenčným spôsobom riešenie pomocou BFS algoritmu pre rýchle rozvetvenie do odlišných medzistavov. Avšak novo vytvorené medzistavy sa ukladajú do poľa medzistavov až pokiaľ v ňom nie je predom určený počet medzistavov. Počet týchto medzistavov v poli bol určený konštante na 1000. Pri lokálnom testovaní neboli pozorované veľké rozdiely pri akomkoľvek počte od 10 do 10 000. Táto jednovláknová časť nezaberá veľké množstvo času keďže pri jednom BFS behu si vlákno z poľa vezme 1 stav a môže do neho pridať až 8 nových stavov. V praxi je z časového hľadiska táto časť takmer zanedbateľná.

Paralelná časť prichádza v DFS riešení kde sa týchto 1000 medzistavov postupne rieši vo for cykle. Tento cyklus je riešený paralelne s dynamickým plánovaním. Voľné vlákno tak vždy dostane najbližší ešte nevyriešený stav, čím sa maximalizuje čas kedy pracujú všetky vlákna:

```
# pragma omp parallel for schedule(dynamic)
   for(const auto& current : dfs_states_vector){
      dfs(current.gameState);
   }
```

Samotná DFS funkcia je identická s tou v sekvenčnom riešení iba s výnimkou sekcii *#pragma omp critical,* ktoré fungujú podobne ako pri taskovom paralelizme.

4 Popis paralelniho algoritmu a jeho implementace v MPI

Implementácia MPI využíva viacero súbežne bežiacich procesov a je typu master-slave. Master proces najprv vytvorí 1000 rôznych medzistavov rovnako ako v dátovom paralelizme a tieto stavy ďalej rozposiela slave procesom. Tie ich riešia paralelným spôsobom rovnako ako prebieha riešenie DFS v taskovom paralelizme.
Najväčšia zmena v algoritme riešenia sa týka posielania práce a riešení medzi master a slave procesmi. Master proces beží v nekonečnej slučke, ktorí sa skladá z 2 častí:
1. Pokiaľ je slave proces označený ako voľný, pošle mu nový medzistav na riešenia. Ak došla práca, zašle ukončovací signál. 2. Ak nie je žiadny slave proces voľný, master čaká na príchod akejkoľvek správy s riešením.
Slave proces aktívne čaká v nekonečnej slučke na príchod správy s novým stavov na riešenie. Tento stav potom okamžite začne riešiť a na konci odošlo správu master procesu s riešením.
Pre šetrenie výpočetného času si master ukladá najlepšiu hodnotu z obdržaných riešení, ktorú potom posiela slave procesom spolu s novým medzistavom.

Kvôli posielaniu správ cez MPI vznikli statické verzie medzistavov DFS a stavu šachovnice, ktoré vymenili c++ vector za pole fixnej dĺžky a funkcie na konverziu medzi

statickou a pôvodnou verziou. Pole s ťahmi na šachovnici má dĺžku 100 a pole s figúrkami jednej farby má dĺžku 50. Obe zvolené konštanty s ukázali ako postačujúce. Ostatné premenné ostali nezmenené.
Na zasielanie každého statického stavu vznikol vlastný MPI struct, ktorý bol následne posielaný ako payload v príkaze MPI_Send a rozbalený cez MPI_Recv. Ešte jeden MPI struct vznikol na zasielanie výsledku.

MPI struct šachovnice:

```
const int nitems1 = 4; // Number of items in the struct
int blocklengths1[4] = {KNIGHTS_SIZE , KNIGHTS_SIZE , 1, 1}; // Number of elements in each block
MPI_Datatype types1[4] = {MPI_INT, MPI_INT, MPI_INT, MPI_INT}; // Datatype of each block
MPI_Datatype gameStateStaticType;
MPI_Aint offsets1[4];

offsets1[0] = offsetof(GameStateStatic, whiteKnights); // Offset of whiteKnights
offsets1[1] = offsetof(GameStateStatic, blackKnights); // Offset of blackKnights
offsets1[2] = offsetof(GameStateStatic, whiteKnightsSize); // Offset of whiteKnightsSize
offsets1[3] = offsetof(GameStateStatic, blackKnightsSize); // Offset of blackKnightsSize
MPI_Type_create_struct(nitems1, blocklengths1, offsets1, types1, &gameStateStaticType);
MPI_Type_commit(&gameStateStaticType);
```

Celkový výsledok sa priebežne ukladá v master procese, ktorý na záver po ukončení všetkých ostatných procesov vypíše výsledok.

5 Namerene vysledky a vyhodnoceni

Všetky 4 verzie programu boli merané na Arm clustri. Na meranie boli vybrané 3 inštancie:

1. In_0010.txt
2. In_0010.txt so zväčšenou šachovnicou zo 4x5 na 5x5
3. In_0016.txt so zväčšenou šachovnicou zo 4x4 na 5x4

Výsledky merania:

Poradové číslo merania	Typ programu	Instancia	Počet procesov	Počet vlákien (celkovo)	Čas (s)	Správny výsledok
1	Sequential	1	1	1	182.129	áno
2	Task	1	1	48	65.5251	áno
3	Task	1	1	16	65.476	áno
4	Task	1	1	32	65.4648	áno
5	Task	1	1	12	65.5003	áno
6	Task	1	1	4	73.8122	áno
7	Task	1	1	2	137.909	áno
8	Task	1	1	1	248.302	áno
9	Data	1	1	48	6.41097	áno
10	Data	1	1	32	8.2452	áno
11	Data	1	1	16	13.6028	áno

12	Data	1	1	12	17.1317	áno
13	Data	1	1	4	49.1952	áno
14	Data	1	1	2	98.1399	áno
15	Data	1	1	1	195.67	áno
16	Mpi	1	4	48	85.9565	áno
17	Mpi	1	3	48	130.727	áno
18	Sequential	2	1	1	517.634	áno
19	Task	2	1	48	176.504	áno
20	Task	2	1	32	175.629	áno
21	Task	2	1	16	175.623	áno
22	Task	2	1	12	175.662	áno
23	Task	2	1	4	212.804	áno
24	Task	2	1	2	379.671	áno
25	Task	2	1	1	700.346	áno
26	Data	2	1	48	17.806	áno
27	Data	2	1	32	23.2242	áno
28	Data	2	1	16	38.4536	áno
29	Data	2	1	12	49.4029	áno
30	Data	2	1	4	144.571	áno
31	Data	2	1	2	288.632	áno
32	Data	2	1	1	578.176	áno
33	Mpi	2	4	48	245.637	áno
34	Mpi	2	3	48	378.232	áno
35	Sequential	3	1	1	201.294	áno
36	Task	3	1	48	75.1737	áno
37	Task	3	1	32	72.0443	áno
38	Task	3	1	16	72.0652	áno
39	Task	3	1	12	72.0788	áno
40	Task	3	1	4	72.6601	áno
41	Task	3	1	2	132.532	áno
42	Task	3	1	1	243.519	áno
43	Data	3	1	48	7.65546	áno
44	Data	3	1	32	8.99952	áno
45	Data	3	1	16	13.6673	áno
46	Data	3	1	12	17.7545	áno
47	Data	3	1	4	50.0752	áno
48	Data	3	1	2	99.1303	áno
49	Data	3	1	1	197.629	áno
50	Mpi	3	4	48	137.75	áno
51	Mpi	3	3	48	202.93	áno

Taskový paralelizmus bol oproti sekvenčnému riešeniu takmer 3-krát rýchlejší pri 4 a viac vláknach. Škálovanie bolo efektívne od 2 po 4 vlákna, od 4 do 48 vlákien boli časy vždy takmer identické.
Dátový paralelizmus s 48 vláknami bol miestami až 30-krát rýchlejší ako sekvenčné riešenie a škáloval sa pomerne dobre s pozorovaným zrýchlením s každým zvýšením počtu vlákien.

Mpi riešenie bolo rýchlejšie ako sekvenčné ale pomalšie ako obe paralelné verzie. Verzia so 4 procesmi a 48 vláknami bola asi 2-krát rýchlejšia ako sekvenčné riešenie. Táto verzia programu sa zdá viac benefituje z väčšieho počtu procesov a veľký počet vlákien naopak nevie využiť. Rozdiel je zrejmý už z tabuľky medzi časmi 3 a 4 procesov a pri lokálnom testovaní bolo pozorované ďalšie výrazné zrýchlenie aj pri 8 a 16 procesoch s rovnakým počtom vlákien.

6 Zaver

Výsledný program skutočne beží vo viacvláknovom režime rýchlejšie, pričom ako najrýchlejšia varianta sa ukázal dátový paralelizmus, ktorý sa zároveň aj veľmi dobre škáloval. Taskový paralelzmus a MPI riešenie boli stále rýchlejšie ako sekvenčné. Práca pre mňa bola prvou skúsenosťou s efektívnym viacvláknovým programovaním. Programovanie v OpenMP a MPI mi prišlo celkom pochopiteľné a nie o moc ťažšie ako bežná práca v C++, čomu celkom určite pomohli prezentácie a výklady na cvičeniach keďže s prehľadom najdlhšie mi trvalo nájsť dobrý návod na prácu so structmi v MPI, ktoré tam asi ako jediná vec neboli. Nová pre mňa bola aj práca na clusteri s posielaním skriptov, s ktorým sa pracuje jednoduchšie ako som očakával. Celkovo som v práci nenarazil na žiadny príliš ťažký bod a vždy sa mi podarilo splniť zadanie, ktoré mi prišlo v priemere tak primerane náročné. (a konečne mám fitvpn)

7 Literatura

[1] Zadane semestrálnej práce na courses:
https://courses.fit.cvut.cz/NI-PDP/labs/zadani-semestralnich-praci.html